PDH System — Math Explainer

FIR Lowpass Design · Controller Metrics · Bode Analysis

This document explains the mathematics behind three parts of the codebase in plain terms. No background in DSP or control theory is assumed.

Contents

1 FIR Lowpass Filter Design . . . 2
- 1.1 What is a lowpass filter? . . . 2
- 1.2 The sinc impulse response . . . 2
- 1.3 Why windowing? . . . 2
- 1.4 Q15 quantization . . . 4
- 1.5 Frequency response . . . 5
- 1.6 How the cocotb testbench verifies the RTL . . . 6

2 Controller Performance Metrics (GUI) . . . 7
- 2.1 Power Spectral Density (Wiener-Khinchin) . . . 7
- 2.2 RMS Tracking Error . . . 7
- 2.3 Settling Time and Overshoot . . . 8
  - 2.3.1 Settling Time . . . 8
  - 2.3.2 Overshoot . . . 8
- 2.4 Control Output Metrics . . . 9
- 2.5 Cross-Correlation: Correction Quality . . . 9

3 Open-Loop Bode Analysis (cocotb) . . . 11
- 3.1 Measuring H(jω) with a DFT . . . 11
- 3.2 Line Delay Correction . . . 11
- 3.3 Phase Unwrapping . . . 12
- 3.4 Stability Margins . . . 12

1 FIR Lowpass Filter Design

Files: `client/pdh_api/fir_design.py`, `core/hw/sim/utils/test_fir_freq_response.py`

The FPGA has a 32-tap FIR (finite impulse response) lowpass filter sitting in the signal path. The Python code designs the coefficients for this filter, and the cocotb testbench verifies that the RTL actually implements them correctly.

1.1 What is a lowpass filter?

A lowpass filter lets slow-varying signals through and blocks fast ones. "Slow" and "fast" are defined by a **corner frequency** f_c: frequencies below f_c pass with roughly unit gain; frequencies above f_c are attenuated.

The ideal lowpass filter — the "brick-wall" filter — has a perfectly sharp cutoff: gain exactly 1 below f_c, gain exactly 0 above it. In the time domain, this ideal filter's impulse response is a sinc function that extends infinitely in both directions. Since we can't implement an infinite filter, we approximate it with a finite number of **taps** (coefficients).

1.2 The sinc impulse response

The 125 MHz FPGA clock gives us a sample rate of $F_s = 125 \cdot 10^6$ Hz. First, we convert the corner frequency to a **normalised digital frequency**:

$$\omega_c = \frac{2\pi f_c}{F_s}$$

This maps f_c to radians per sample, with the Nyquist frequency sitting at π rad/sample. Now we can write down the ideal lowpass impulse response, evaluated at each tap index $n \in \{0, 1, ..., N-1\}$:

$$h[n] = \frac{\sin(\omega_c(n - n_c))}{\pi(n - n_c)}$$

where $n_c = \frac{N-1}{2}$ is the **centre tap index**. When $n = n_c$ the denominator is zero. Applying L'Hôpital's rule (or just taking the limit) gives the centre-tap value:

$$h[n_c] = \frac{\omega_c}{\pi}$$

Notice that $\frac{\omega_c}{\pi}$ equals $2\frac{f_c}{F_s}$, so for a corner at $f_c = 1$ MHz and $F_s = 125$ MHz the centre tap value is $2 \times \frac{10^6}{125} \times 10^6 \approx 0.016$.

In the code (`fir_design.py` line 29):

```
x  = n - nc
h  = wc / math.pi if abs(x) < 1e-10 else math.sin(wc * x) / (math.pi * x)
```

1.3 Why windowing?

If you just truncate the sinc to N taps and use it as-is, you get **Gibbs ripple** — oscillations in the passband and stopband caused by the abrupt cut at both ends of the coefficient array. The fix is to multiply every tap by a **window function** $w[n]$ that smoothly tapers from near-zero at the edges to its maximum at the centre.

This softens the truncation at the cost of a slightly less sharp rolloff. Different window shapes offer different trade-offs between transition width and stopband rejection:

Window	**Formula $w[n]$**	**Transition**	**Stopband**
Rectangular	1	Sharpest	Worst ($\approx$ 21 dB)
Hann	$0.5 - 0.5\cos\left(\frac{2\pi n}{N-1}\right)$	Moderate	$\approx$ 44 dB
Hamming	$0.54 - 0.46\cos\left(\frac{2\pi n}{N-1}\right)$	Moderate	$\approx$ 53 dB
Blackman	$0.42 - 0.50\cos\left(\frac{2\pi n}{N-1}\right) + 0.08\cos\left(\frac{4\pi n}{N-1}\right)$	Widest	Best ($\approx$ 74 dB)

The final tap coefficient is the product of the sinc value and the window:

$$h_{\text{final}[n]} = h[n] \cdot w[n]$$

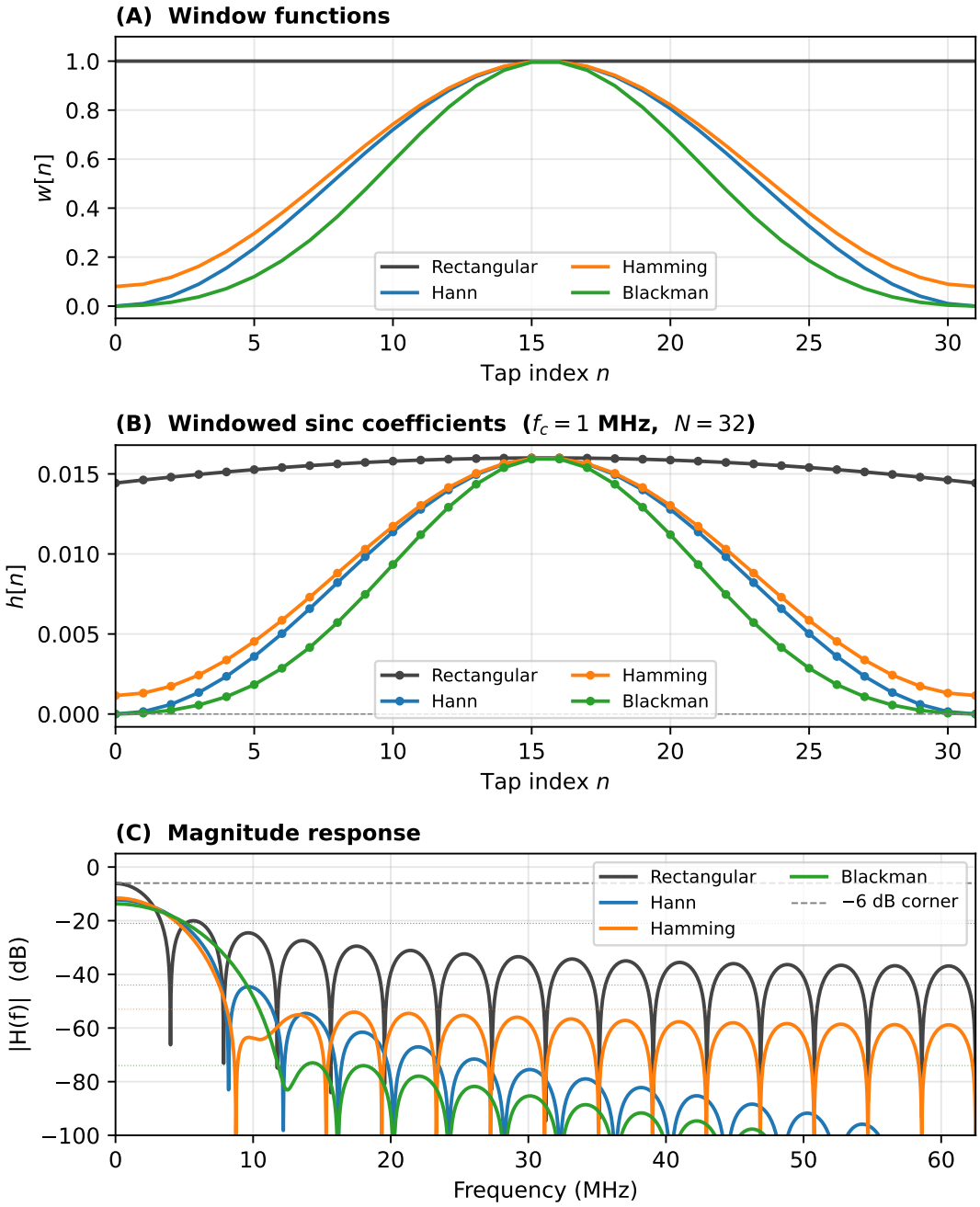

Figure 1: **FIR window comparison. (A)** The four window shapes $w[n]$ on $n = 0...31$. **(B)** Resulting windowed-sinc tap coefficients ($f_c = 1$ MHz, $N = 32$) — the rectangular window leaves large outer taps, while Blackman tapers them almost to zero. **(C)** Magnitude responses in dB: the rectangular window has the sharpest transition but the shallowest stopband floor (21 dB), while Blackman achieves 74 dB at the cost of a wider rolloff. The −6 dB point on every curve coincides with $f_c = 1$ MHz.

1.4 Q15 quantization

The FPGA works in 16-bit fixed-point arithmetic. The coefficient format is **Q15**: a number in the range $(-1, 1)$ is represented as a signed 16-bit integer by:

$$h_{\text{Q15}} = \text{round}\Big(h_{\text{final}[n]} \times 32768\Big)$$

clamped to $[-32768, 32767]$. Q15 can only represent numbers strictly less than 1 in magnitude, so the code checks that $\max_n |h_{\text{final}[n]}| < 1$ before quantising.

For even N (like our $N = 32$), $n_c = (N-1)/2 = 15.5$ is not an integer, so no tap sits exactly at n_c. The two taps closest to the centre are $n = 15$ and $n = 16$, both at distance $1/2$ from n_c:

$$h[15] = h[16] = \frac{\sin(\omega_c/2)}{\pi \cdot (1/2)} = \frac{2\sin(\omega_c/2)}{\pi}$$

Because $\sin(x) < x$ for $x > 0$, this is strictly less than $2(\omega_c/2)/\pi = \omega_c/\pi$. And since the filter's corner must be below Nyquist, $f_c < F_s/2$, we have $\omega_c < \pi$, so $\omega_c/\pi < 1$. The actual maximum tap $2\sin(\omega_c/2)/\pi$ is bounded above by $\omega_c/\pi < 1$, and any window (≤ 1 everywhere) can only reduce it further — so all taps are Q15-safe. The explicit code check is a belt-and-suspenders guard against unusual parameter combinations.

1.5 Frequency response

Given the final tap array $h[n]$, the filter's frequency response at any frequency f is the Discrete-Time Fourier Transform (DTFT) evaluated at $\omega = 2\pi f/F_s$:

$$H(f) = \sum_{n=0}^{N-1} h[n] \cdot e^{-j\omega n}$$

$H(f)$ is a complex number. Its magnitude $|H(f)|$ is the gain at that frequency, and its angle $\angle H(f)$ is the phase shift introduced.

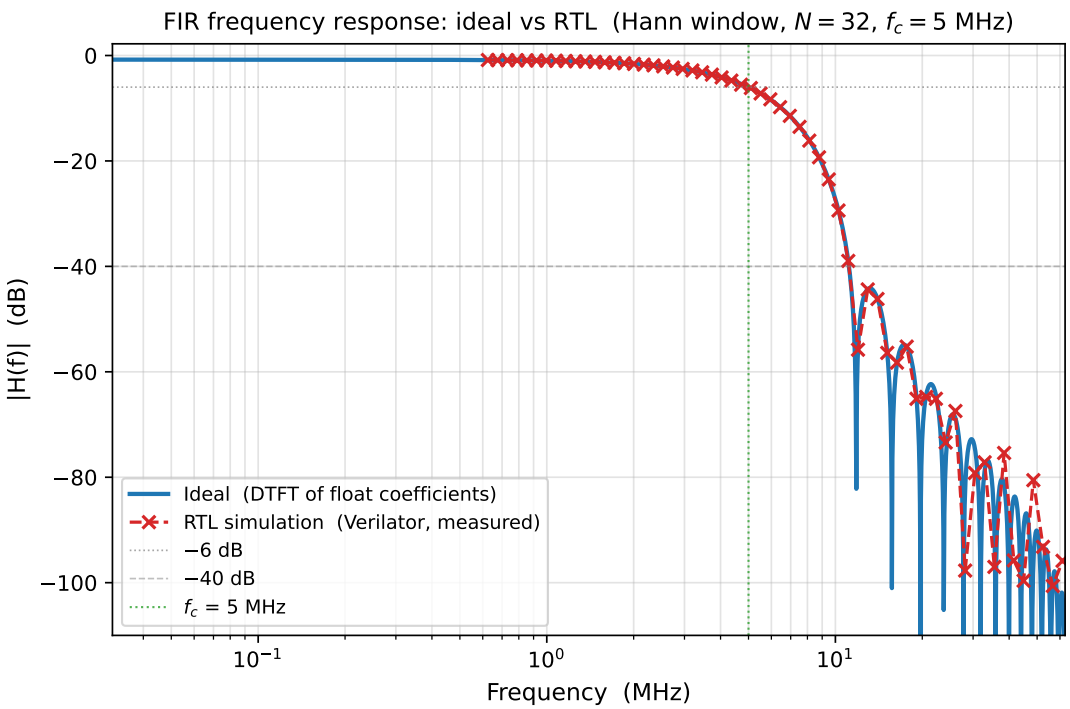

Figure 2: **Ideal vs RTL frequency response** (Hann window, $N = 32$, $f_c = 5$ MHz). The solid blue curve is the ideal response computed analytically from the float-precision coefficients using the DTFT formula. The red dashed curve with crosses is the response actually measured by the cocotb/Verilator RTL simulation: a cosine was driven into `din_i` at each frequency and the steady-state output amplitude was read from `dout_o`. The two curves overlap almost exactly, confirming that the RTL correctly implements the designed filter. Any residual deviation at deep stopband nulls is due to Q15 coefficient quantisation in the FPGA.

Because the tap array is symmetric ($h[n] = h[N-1-n]$), the filter has **linear phase** — the phase shift is exactly proportional to frequency, meaning the filter introduces a pure time delay with no phase distortion. With $N = 32$ (even number of taps) this is a **Type II** linear-phase FIR, which has a mathematically forced zero at exactly $\omega = \pi$ (the Nyquist frequency). For a lowpass filter this is fine — we want zero gain at Nyquist anyway.

1.6 How the cocotb testbench verifies the RTL

`test_fir_freq_response.py` checks that the actual silicon (well, simulated RTL) matches the design formula. The procedure is:

1. **Load coefficients** into the FIR BRAM via the tap interface.
2. **Wait for the pipeline to flush.** The FIR has a latency of $N_{\text{taps}} + \lceil \log_2(N_{\text{taps}}) \rceil + 1 + 16$ clock cycles (one register per tap output + adder-tree depth + margin). For 32 taps this is $32 + 5 + 1 + 16 = 54$ cycles.
3. **Sweep frequencies.** For each test frequency f:
   - Drive the input with a cosine: $x[n] = A\cos(\omega n)$
   - Collect the steady-state output $y[n]$
   - Estimate the amplitude of each signal using a single-bin DFT:

     $$A_{\text{sig}} = \frac{2}{M}\left|\sum_{n=0}^{M-1} s[n] \cdot e^{-j\omega n}\right|$$

     The factor of 2 appears because the DFT of a real cosine $A\cos(\omega n)$ concentrates half its energy ($AM/2$) at positive frequency and half at negative frequency; evaluating only at the positive bin gives $AM/2$, so multiplying by $2/M$ recovers A.
   - Compute $|H_{\text{RTL}(f)}| = \frac{A_{\text{out}}}{A_{\text{in}}}$
4. **Compare** to the ideal $|H(f)|$ computed analytically from the float coefficients using the DTFT formula above.

If the RTL is correct the two curves lie on top of each other.

2 Controller Performance Metrics (GUI)

File: `client/pdh_api/api.py` — function `api_control_metrics()`

When you click **Compute Control Metrics** in the GUI, the ARM server asks the FPGA to DMA a capture buffer containing N time-domain samples of three signals: `pid_in` (filter output / PID input), `err` (error = setpoint − PID input), and `pid_out` (DAC drive). The Python code then computes several metrics from this data.

The effective sample rate is $f_s = F_s/$ decimation Hz (the PID's internal decimation reduces the sample rate).

2.1 Power Spectral Density (Wiener-Khinchin)

The PSD tells you how much power is in each frequency band of a signal. Rather than taking an FFT of the raw signal directly (which has spectral leakage problems for short records), the code uses the **Wiener-Khinchin theorem**: the PSD is the Fourier transform of the **autocorrelation** of the signal.

Step 1 — Remove the mean:

$$\tilde{x}[n] = x[n] - \bar{x}$$

Step 2 — Compute the full (biased) autocorrelation:

$$R[k] = \sum_{n=0}^{N-1-|k|} \tilde{x}[n] \cdot \tilde{x}[n+k], \quad k \in \{-(N-1), ..., N-1\}$$

This is a length-$(2N-1)$ sequence that measures how similar the signal is to a shifted copy of itself. `numpy.correlate(x, x, mode='full')` computes exactly this (no $1/N$ normalisation — this is the **biased** estimator).

Step 3 — FFT and normalise:

$$S(f_j) = \frac{|\mathrm{RFFT}(R)[j]|}{f_s \cdot N}$$

The result $S(f_j)$ has units of counts^2 per Hz. Integrating S over the positive-frequency range $[0, f_s/2]$ gives the **two-sided spectral mass** — approximately half the signal variance (because the two-sided PSD is symmetric and only the positive half is stored).

Frequency axis: because R has length $2N-1$, the rfft returns N frequency bins spanning 0 to $(N-1)f_s/(2N-1) \approx f_s/2$.

The GUI also plots the **centre of mass** of each PSD:

$$f_{\mathrm{com}} = \frac{\int f \cdot S(f) df}{\int S(f) df}$$

This tells you where most of the noise energy sits in frequency. The optional "mass" annotation shows $\int_0^{f_s/2} S(f) df$, which is a useful relative number for comparing captures (not the total signal power, which would require also integrating the mirrored negative-frequency half).

2.2 RMS Tracking Error

The root-mean-square error is simply:

$$\mathrm{RMS}_{\mathrm{err}} = \sqrt{\frac{1}{N}\sum_{n=0}^{N-1} e[n]^2}$$

in ADC counts. Smaller is better — it directly quantifies how far the laser frequency is wandering from the setpoint.

2.3 Settling Time and Overshoot

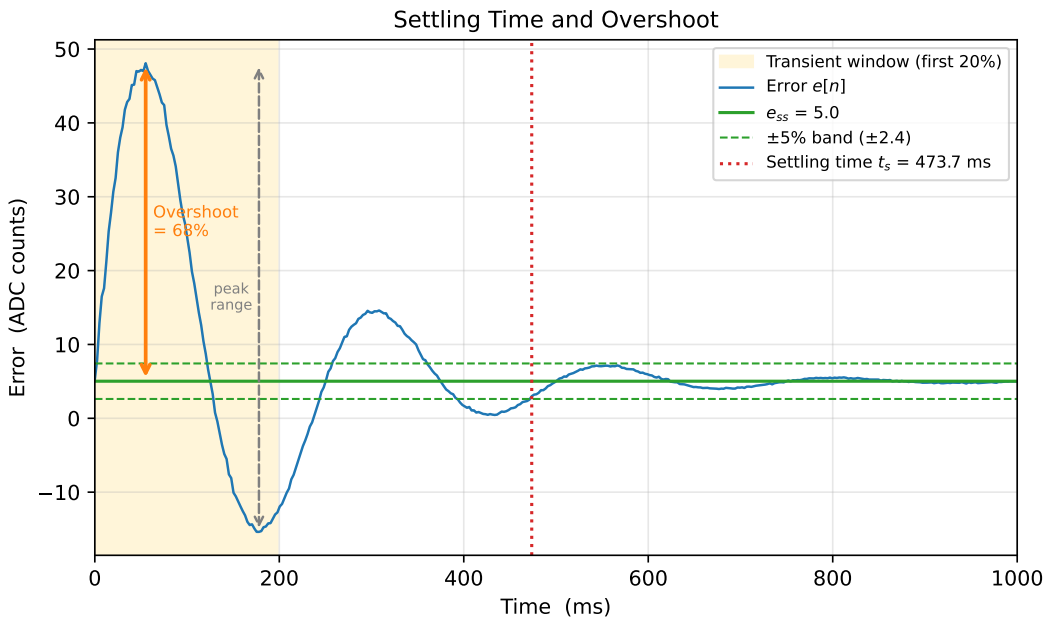

Figure 3: **Settling time and overshoot, illustrated on a synthetic error signal.** The shaded region is the first 20% of the capture (transient window used for overshoot). The green lines show e_{ss} and the ±5% tolerance band. The red dashed line marks the settling time t_s. The orange double-headed arrow shows the overshoot relative to e_{ss}; the grey bracket shows the full transient range used in the denominator.

2.3.1 Settling Time

This is the time from the start of the capture until the error signal has stopped doing anything interesting.

1. Estimate the **steady-state value** as the mean of the last 20% of samples: $e_{\mathrm{ss}} = \mathrm{mean}(e[0.8N : N])$
2. Compute the **tolerance band** as ±5% of the peak absolute error: $\mathrm{band} = 0.05 \cdot \max_n |e[n]|$
3. Find the earliest sample index i such that $|e[n] - e_{\mathrm{ss}}| \leq \mathrm{band}$ for **all** $n \geq i$
4. Settling time $= i/f_s$ seconds

If the error never fully settles (common during initial lock acquisition), the metric is reported as `NaN`.

2.3.2 Overshoot

Overshoot is evaluated only over the first 20% of the capture, which captures the initial transient before the loop settles.

$$\mathrm{Overshoot} = \frac{\max(e_{\mathrm{transient}}) - e_{\mathrm{ss}}}{\max(e_{\mathrm{transient}}) - \min(e_{\mathrm{transient}})} \times 100\%$$

The denominator normalises by the full transient range rather than the steady-state value, making the number meaningful even when $e_{\text{ss}} \approx 0$.

2.4 Control Output Metrics

These measure how hard the PID is working:

Metric	Formula / Meaning
Ctrl RMS	$\sqrt{\text{mean}(u[n]^2)}$ — average drive level
Ctrl Max	$\max\lvert u[n]\rvert$ — worst-case excursion
Ctrl P95	95th percentile of $\lvert u[n]\rvert$ — robust maximum (ignores outliers)
Ctrl Slew RMS	$\sqrt{\text{mean}((u[n]-u[n-1])^2)}$ — how fast the output is changing sample-to-sample

where $u[n]$ is `pid_out` in DAC counts. High slew RMS relative to Ctrl RMS means the output is jittering rapidly; a ratio close to zero means the output is changing slowly and smoothly.

2.5 Cross-Correlation: Correction Quality

The cross-correlation function (CCF) between `pid_out` and `err` answers the question: **does the controller output actually predict future error reduction, and with what delay?**

Step 1 — Zero-mean the signals:

$$\tilde{u}[n] = u[n] - \bar{u}, \quad \tilde{e}[n] = e[n] - \bar{e}$$

Step 2 — Compute the normalised cross-correlation:

$$\text{CCF}[k] = \frac{1}{\sigma_u \cdot \sigma_e \cdot N} \sum_{n=0}^{N-1} \tilde{u}[n] \cdot \tilde{e}[n+k]$$

where σ_u, σ_e are the standard deviations. By Cauchy-Schwarz, $|\text{CCF}[k]| \leq 1$ for all lags $k \in \{-(N-1), ..., N-1\}$.

Positive k means "pid_out at time n correlates with err at future time $n+k$" — i.e., the controller acts first and the error responds k samples later.

Interpreting the result:

- **CCF peak magnitude** — a value near ± 1 means the controller output is strongly correlated with the error signal (good correction). Near 0 means the output and error are essentially uncorrelated (poor or absent control).
- **Lag at the peak** (k_{peak}/f_s in seconds) — the apparent loop delay. This is how long after the PID acts before the error responds.

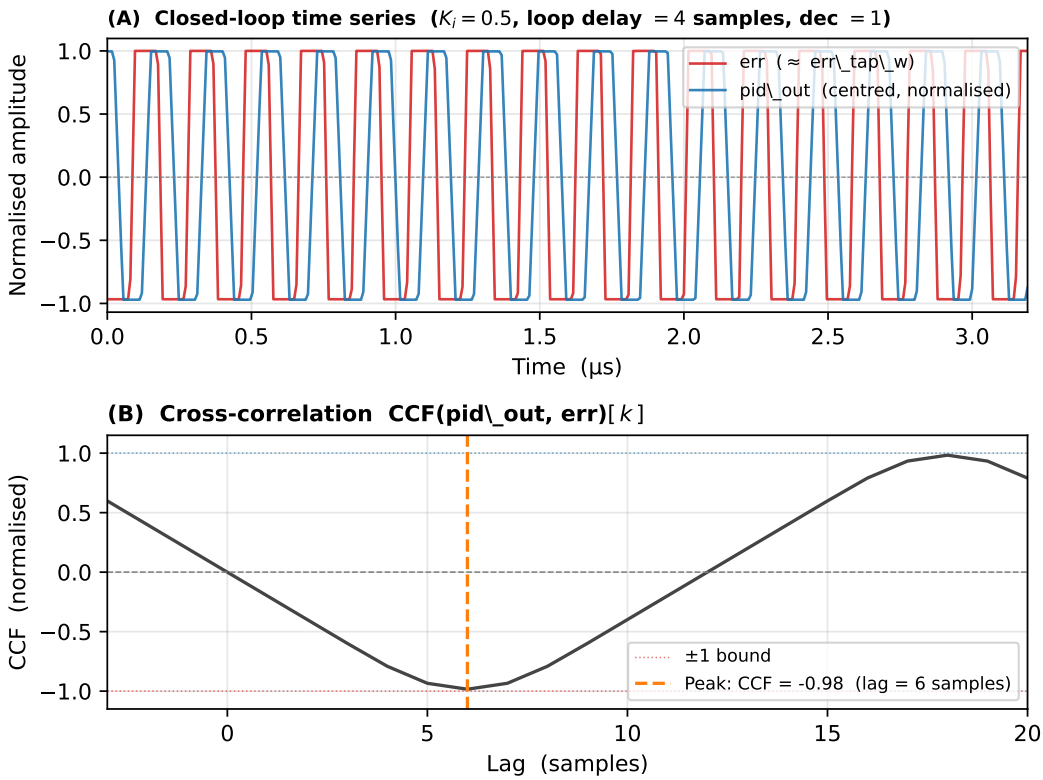

Figure 4: **Cross-correlation between `pid_out` and `err`. (A)** Synthetic time series: the controller fires a burst and the error dips after an 8 ms loop delay. **(B)** The normalised CCF peaks at lag 8 ms, directly reading out the loop delay from the data.

3 Open-Loop Bode Analysis (cocotb)

File: `core/hw/sim/utils/test_bode.py`

The Bode test measures the **open-loop transfer function** of the controller RTL. The idea: inject a sine wave at the plant input (ADC), measure what comes out at the controller output (DAC), and take their ratio. Repeating this at many frequencies gives the full Bode plot.

3.1 Measuring H(jω) with a DFT

For a single test frequency f (angular frequency $\omega = 2\pi f/F_s$):

Drive the ADC with a cosine in offset-binary format (the Red Pitaya ADC outputs 14-bit unsigned values centred at 8192):

$$\text{raw}[n] = 8192 + A\cos(\omega n)$$

Centre the signals to remove the DC offset. The RTL also negates the ADC input (a design convention), so the effective input seen by the PID is:

$$x_{\text{in}[n]} = -(\text{raw}[n] - 8192) = -A\cos(\omega n)$$

The DAC output is similarly recentred:

$$x_{\text{out}[n]} = \text{dac_dat_o}[n] - 8191$$

Compute the DFT at exactly frequency f for each signal (using M steady-state samples):

$$X_{\text{in}} = \frac{1}{M}\sum_{n=0}^{M-1} x_{\text{in}[n]} \cdot e^{-j\omega n}, \quad X_{\text{out}} = \frac{1}{M}\sum_{n=0}^{M-1} x_{\text{out}[n]} \cdot e^{-j\omega n}$$

Transfer function estimate:

$$H_{\text{RTL}(j\omega)} = \frac{X_{\text{out}}}{X_{\text{in}}}$$

The $1/M$ normalisation appears in both numerator and denominator and cancels in the ratio — any consistent normalisation gives the same result.

This is a complex number. Its magnitude $|H|$ is the gain at that frequency; its angle $\angle H$ is the phase shift.

Using a single-bin DFT (evaluating at exactly ω) avoids spectral leakage entirely — it works because the input **is** a pure sinusoid at that exact frequency.

3.2 Line Delay Correction

In the real lab, the signal has to travel from the DAC output back to the ADC input through optical and electronic components. This round-trip introduces a time delay τ that adds phase lag without being present in the RTL simulation.

We correct for it analytically by multiplying by a pure phase-delay factor:

$$H(j\omega) = H_{\text{RTL}(j\omega)} \cdot e^{-j2\pi f\tau}$$

Multiplying a complex number by $e^{-j\theta}$ rotates it clockwise in the complex plane by θ radians, without changing its magnitude. The diagram below makes this geometric interpretation clear.

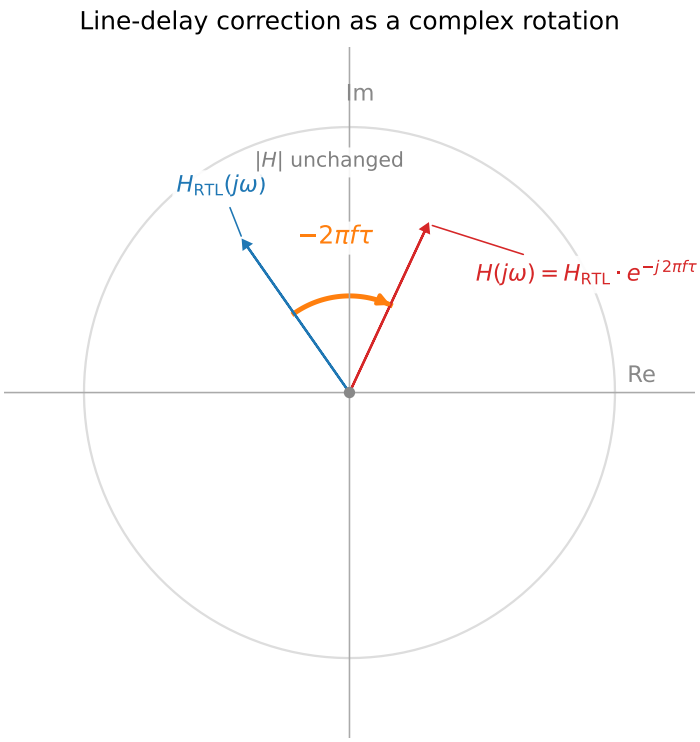

Figure 5: **Line delay as a rotation in the complex plane.** The blue arrow is $H_{\mathrm{RTL}(j\omega)}$ for a single frequency. The red arrow is the corrected value after multiplying by $e^{-j2\pi f\tau}$ — same length ($|H|$ unchanged), rotated clockwise by $2\pi f\tau$ radians (orange arc).

3.3 Phase Unwrapping

The raw phase from `cmath.phase()` (or `numpy.angle()`) is **wrapped** to the interval $(-\pi, \pi]$. When the actual phase crosses $-\pi$, the measured value jumps discontinuously by $+2\pi$.

Unwrapping fixes this by scanning adjacent frequency points and adding or subtracting 2π whenever a jump larger than π is detected:

```
for i in range(1, len(phases)):
    diff = phases[i] - phases[i-1]
    while diff >  pi: diff -= 2*pi
    while diff < -pi: diff += 2*pi
    phases[i] = phases[i-1] + diff
```

After unwrapping the phase curve is continuous, which is necessary for correctly identifying where the phase crosses $-180°$.

3.4 Stability Margins

Two numbers summarise how close the loop is to instability.

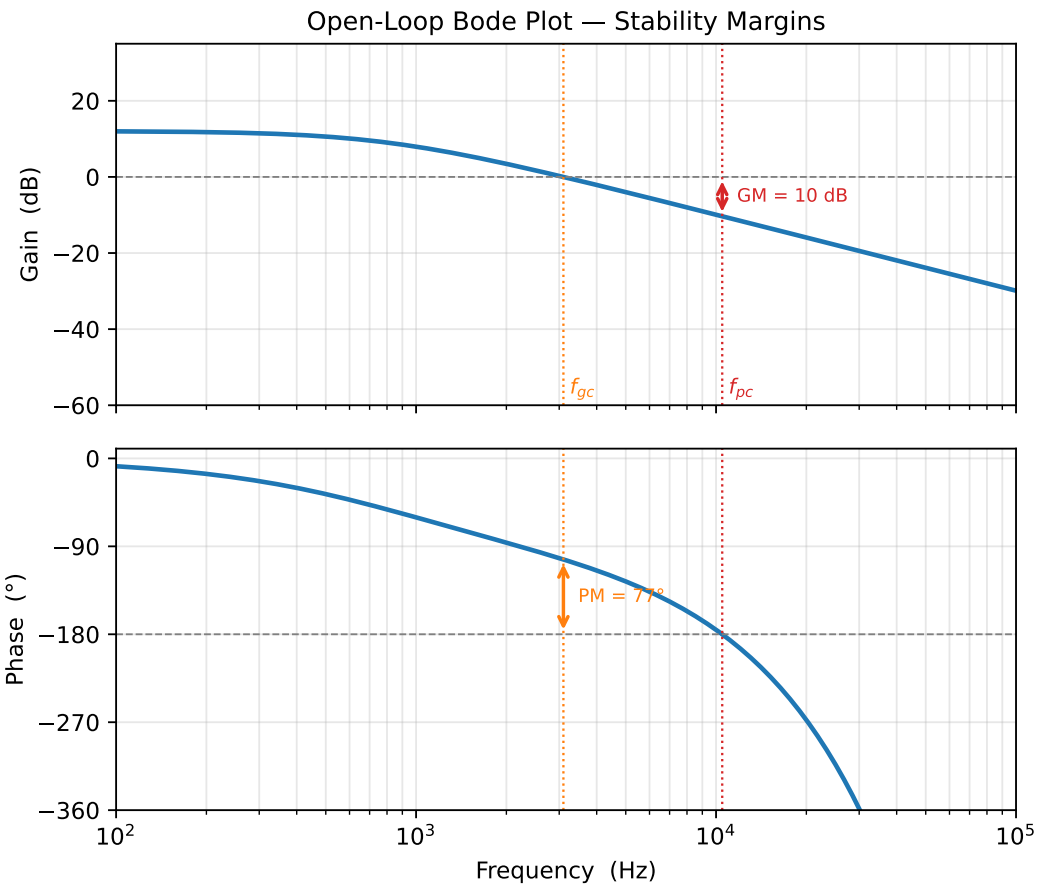

Figure 6: **Bode plot with stability margins annotated. Top:** magnitude in dB. The orange dashed vertical line marks the gain crossover frequency f_{gc} (where $|H| = 0$ dB); the orange double-headed arrow on the phase plot is the Phase Margin (PM). **Bottom:** unwrapped phase in degrees. The red dashed vertical line marks the phase crossover frequency f_{pc} (where $\angle H = -180°$); the red double-headed arrow on the gain plot is the Gain Margin (GM). Both margins are positive here, indicating a stable loop.

Gain crossover frequency f_{gc}: the frequency where the loop gain drops to exactly 1 (0 dB). The test finds this by scanning adjacent frequency pairs and interpolating linearly when $|H|$ crosses 1 going downward.

Phase margin (PM): at f_{gc}, how far is the phase from the critical value of $-180°$?

$$\mathrm{PM} = 180° + \angle H\big(f_{\mathrm{gc}}\big)$$

A positive PM means the loop is stable. Rule of thumb: PM $> 30°$ gives comfortable stability; PM $> 45°$ is considered robust.

Phase crossover frequency f_{pc}: the frequency where the unwrapped phase first reaches $-180°$. Found by the same linear-interpolation scan, watching for $\angle H$ crossing $-180°$ going downward.

Gain margin (GM): at f_{pc}, how much would you have to increase the loop gain before the system goes unstable?

$$\mathrm{GM} = -20\log_{10}\big|H\big(f_{\mathrm{pc}}\big)\big| \quad \mathrm{dB}$$

Positive GM means the gain is below 1 at the phase crossover, so there is headroom before instability.

Quantity	Target	Meaning if violated
Phase Margin	> 30–$45°$	Oscillatory or unstable step response
Gain Margin	> 6 dB	Small gain changes will cause instability

Both margins are found by the same approach: scan the frequency array, find the interval where the relevant quantity crosses its threshold, and interpolate to get the exact crossing frequency.

Generated from codebase sources: `client/pdh_api/fir_design.py`, `client/pdh_api/api.py`, `core/hw/sim/utils/test_fir_freq_response.py`, `core/hw/sim/utils/test_bode.py`.